Assignment 15:

```
/**
 *
================================================================================
 * KBTCOE Connect — Persistent REST API Server (server.js)
 * Assignment 15: REST API and Database
 *
================================================================================
 */

'use strict';

const path = require('path');
const express = require('express');
require('dotenv').config();

// Configuration & Database Pool
const { checkConnection } = require('./config/database');

// Middleware
const notFound = require('./middleware/notFound');
const errorHandler = require('./middleware/errorHandler');

// Routers
const opportunitiesRouter = require('./routes/opportunities');
const authRouter = require('./routes/auth');
const postsRouter = require('./routes/posts');

// Initialize application
const app = express();
const PORT = process.env.PORT || 3000;

//
================================================================================
// 1. Global Middleware
//
================================================================================

/**
 * Custom Request Logger Middleware.
 * Omits passwords and authorization tokens to preserve security in server logs.
 */
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
    const duration = Date.now() - start;
    const logTime = new Date().toISOString().replace('T', ' ').slice(0, 19);
    console.log(
      `[${logTime}] ${req.method} ${req.originalUrl} -> ${res.statusCode}
(${duration}ms)`
    );
```

```
  });
  next();
});

// Parse JSON request payloads
app.use(express.json());

// Parse URL-encoded bodies
app.use(express.urlencoded({ extended: true }));

// Serve frontend demonstration client
app.use(express.static(path.join(__dirname, 'public')));

//
================================================================================
// 2. Health Endpoint
//
================================================================================

/**
 * GET /api/health
 * Returns service status and verifies database connection health.
 */
app.get('/api/health', async (req, res) => {
  const dbHealth = await checkConnection();
  const uptimeSeconds = Math.floor(process.uptime());

  res.status(dbHealth.isConnected ? 200 : 503).json({
    success: dbHealth.isConnected,
    message: dbHealth.isConnected
      ? 'KBTCOE Connect REST API & Database are healthy and operational'
      : 'API is running but relational database is unreachable',
    data: {
      status: dbHealth.isConnected ? 'UP' : 'DEGRADED',
      service: 'KBTCOE Connect Persistent REST API',
      version: '1.0.0',
      database: {
        connected: dbHealth.isConnected,
        engine: 'MySQL / MariaDB',
        version: dbHealth.version || 'Unknown',
        schema: dbHealth.database || 'kbtcoe_connect_api'
      },
      uptimeSeconds,
      serverTime: new Date().toISOString()
    }
  });
});

//
================================================================================
// 3. Mount Resource Routes
//
================================================================================
```

```
app.use('/api/opportunities', opportunitiesRouter);
app.use('/api/auth', authRouter);
app.use('/api/posts', postsRouter);

//
================================================================================
// 4. Error Handling
//
================================================================================

app.use(notFound);
app.use(errorHandler);

//
================================================================================
// 5. Server Lifecycle Management
//
================================================================================

let server = null;

if (require.main === module) {
  server = app.listen(PORT, async () => {

console.log('================================================================');
    console.log('🚀 KBTCOE Connect Persistent REST API Server is running!');
    console.log(`📡 Base URL:          http://localhost:${PORT}`);
    console.log(`🏥 Health Check:      http://localhost:${PORT}/api/health`);
    console.log(`🗄️ Opportunities API:
http://localhost:${PORT}/api/opportunities`);
    console.log(`🔐 Auth API:          http://localhost:${PORT}/api/auth`);
    console.log(`💬 Posts API:         http://localhost:${PORT}/api/posts`);
    console.log(`🎨 Test UI Web App:   http://localhost:${PORT}/`);

    // Verify DB on boot
    const dbTest = await checkConnection();
    if (dbTest.isConnected) {
      console.log(`🟢 Database:          Connected to ${dbTest.database}
(${dbTest.version})`);
    } else {
      console.log(`🔴 Database:          Connection failed (${dbTest.error})`);
    }

console.log('================================================================');
  });

  const shutdown = (signal) => {
    console.log(`\n[SHUTDOWN] Received ${signal}. Closing server...`);
    server.close(() => {
      console.log('[SHUTDOWN] HTTP server closed gracefully. Exiting.');
      process.exit(0);
    });
  };
```

```
  process.on('SIGINT', () => shutdown('SIGINT'));
  process.on('SIGTERM', () => shutdown('SIGTERM'));
}

module.exports = { app, server, PORT };
```